

Robust CNN-Based Image Classification under Memory Constraints

Aprajita Agarwal (BT2024258)

1 Overview

This report describes the design, training, and evaluation of a lightweight CNN-based vehicle classifier. The objective is to classify real-world street images into five categories: **Bus**, **Truck**, **Car**, **Bike**, and **None** (no vehicle present), subject to a strict **5 MB model size constraint**. The final model achieves **95.03% validation accuracy** with a file size of **4.14 MB**.

2 Dataset

2.1 Source

All training images were sourced from the **ImageNet Winter 2021** dataset. Specific synsets were selected for each class to maximise visual distinctiveness and real-world relevance.

2.2 Synset Selection

A key design decision was the careful selection and exclusion of synsets to reduce inter-class confusion, particularly between **Car** and **Truck**. Several ambiguous synsets were deliberately excluded.

Table 1: ImageNet Synsets Used Per Class

Class	Synset ID	Description
Bus	n02924116	bus, autobus, coach, double-decker, omnibus
Truck	n04467665	trailer truck, tractor trailer, articulated lorry
	n03930630	pickup truck
Car	n02958343	car, automobile, motorcar
	n03100232	convertible
	n04285008	sports car
Bike	n03790512	motorcycle, motorbike
	n02835271	bicycle, tandem bicycle
None	Mixed	Street scenes, pedestrians, buildings (Google Street View)

2.3 Excluded Synsets and Reasoning

Several synsets provided in the assignment specification were intentionally excluded after analysis:

Table 2: Excluded Synsets and Reasons

Synset ID	Description	Reason for Exclusion
n03770679	Minivan	Visually similar to vans and trucks; caused Car/Truck confusion
n03594945	Jeep, Landrover	Resembles pickup trucks and SUVs; ambiguous boundary
n04037443	Race car	Unrealistic for street images; very different from normal cars
n04461696	Tow truck	Often contains a car in the image; confused the model

2.4 Dataset Split

The **None** class was constructed from street scene images (roads, sidewalks, pedestrians, buildings) sourced from Google Street view dataset to match real-world test conditions.

Table 3: Final Dataset Distribution

Class	Train	Val	Total
Bus	2124	530	2654
Truck	2196	548	2744
Car	2160	540	2700
Bike	2076	518	2594
None	2080	520	2600
Total	10636	2656	13292

The dataset is well-balanced across all five classes, with a maximum inter-class difference of approximately 150 images in the training split. An 80/20 train/validation split was applied using random shuffling per class.

3 Model Architecture

3.1 Backbone Selection

MobileNetV3 Small was selected as the backbone. It was designed using Neural Architecture Search (NAS) specifically targeting mobile and CPU inference, incorporating Squeeze-and-Excitation blocks that allow the network to focus on the most discriminative features per channel. Pretrained ImageNet weights were used as the starting point, enabling the model to leverage rich low-level feature representations (edges, textures, shapes) without training from scratch. MobileNetV3 Small provides better accuracy–efficiency tradeoff than MobileNetV2 under tight memory constraints.

3.2 Classifier Head

The pretrained backbone’s original classifier was replaced with a custom 5-class head:

```
model.classifier = nn.Sequential(
    nn.Linear(576, 128),
    nn.Hardswish(),
    nn.Dropout(0.2),
    nn.Linear(128, 5)
)
```

The input dimension of 576 matches the MobileNetV3 Small feature output. **Hardswish** is the activation used natively in MobileNetV3. Dropout of 0.2 reduces overfitting.

4 Training Methodology

4.1 Two-Phase Training

Training was performed in two phases to balance feature extraction stability with fine-grained adaptation:

Table 4: Two-Phase Training Strategy

Phase	Epochs	Description
Phase 1	1–10	Backbone frozen, only classifier head trained, lr = 0.001
Phase 2	11–40	Full model unfrozen, fine-tuned end-to-end, lr = 0.0001

Freezing the backbone in Phase 1 prevents overwriting pretrained ImageNet features before the classifier head has learned meaningful representations. Phase 2 allows the entire network to adapt to the specific visual characteristics of street vehicles.

4.2 Training Configuration

Table 5: Hyperparameters

Parameter	Value
Optimiser	Adam
Phase 1 LR	0.001
Phase 2 LR	0.0001
LR Scheduler	StepLR (step=5, $\gamma=0.5$)
Loss Function	CrossEntropyLoss
Batch Size	32
Max Epochs	40
Early Stopping	Patience = 7
Input Resolution	224×224
Device	Apple M4 (MPS)

4.3 Implementation Details

Table 6: Libraries Used

Library	Purpose
PyTorch 2.x	Model training and inference
TorchVision	MobileNetV3 pretrained weights and transforms
Pillow	Image loading and preprocessing
tqdm	Training progress bar

4.4 Data Augmentation

Strong augmentation was applied to improve robustness to the specific conditions mentioned in the assignment: illumination variations and occlusions.

- **RandomResizedCrop(224, scale=0.7–1.0)** — simulates vehicles at different distances from the camera, forcing the model to recognise partial and zoomed-out views.
- **RandomHorizontalFlip** — accounts for left/right orientation variation of vehicles on roads.
- **ColorJitter(brightness=0.5, contrast=0.5, saturation=0.4, hue=0.1)** — directly targets the illumination variation requirement; simulates different times of day, weather, and lighting conditions.
- **RandomGrayscale(p=0.1)** — handles extreme lighting scenarios and cameras with reduced colour fidelity.
- **RandomRotation(15°)** — accounts for camera tilt and slight perspective variation.
- **GaussianBlur(kernel=3)** — simulates motion blur and images of distant or fast-moving vehicles.
- **RandomErasing(p=0.4, scale=0.02–0.3, ratio=0.3–3.3)** — directly targets the occlusion requirement; randomly blacks out rectangular patches of varying shapes, simulating objects partially obscuring the vehicle.

5 Inference

The `VehicleClassifier` class performs CPU-only inference as required. The preprocessing pipeline matches the validation transform used during training:

```
self.transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.5]*3, [0.5]*3)
])
```

Resizing all inputs to 224×224 ensures the model handles any input resolution cleanly, from the 32×32 minimum to the 256×256 maximum specified in the assignment.

6 Results

Table 7: Final Results Summary

Metric	Value
Best Validation Accuracy	95.03%
Final Model Size	4.14 MB
Size Limit	5.00 MB
Inference Device	CPU
Input Resolution Support	32×32 to 256×256

6.1 Key Observations

- **Convergence** occurred at **epoch 38** out of a maximum of 40, with early stopping patience of 7 confirming the model had fully converged.
- **Phase 2 unfreeze** caused a noticeable accuracy jump at epoch 11, confirming the benefit of end-to-end fine-tuning after the classifier head converges.

7 File Submission

Table 8: Submitted Files

File	Description
<code>vehicle_classifier.py</code>	Inference class with MobileNetV3 Small backbone
<code>student_model.pth</code>	4.14 MB
<code>train_model.py</code>	Full training script with two-phase fine-tuning
<code>trim.py</code>	Utility to trim synset folders to a target image count
<code>split.py</code>	Utility to split dataset into 80/20 train/val structure
<code>report.pdf</code>	This report
<code>README.md</code>	Setup and usage instructions

8 Limitations

- Some vehicles are hard to tell apart. Pickup trucks and large SUVs can look like either a Car or a Truck depending on the angle.
- The None class sometimes has vehicles in the background, which can confuse the model.
- The augmentation used during training cannot cover every real-world scenario. Night-time images, rain, or unusual occlusions may still cause wrong predictions.
- The 5 MB size limit means a smaller model was used, which naturally has less capacity to handle difficult edge cases compared to larger models.

9 Conclusion

A robust vehicle classifier was designed and trained under strict memory constraints. The key contributions are: (1) careful synset curation to eliminate inter-class confusion, particularly between Car and Truck, (2) two-phase transfer learning with MobileNetV3 Small, (3) strong augmentation pipeline directly targeting the assignment’s illumination variation and occlusion requirements. The final model achieves **95.03% validation accuracy** at **4.14 MB**, well within the constraint. Remaining failure cases are primarily driven by inherent visual ambiguity between certain vehicle types and limitations of rectangular occlusion simulation.